

CilialO Fine Tuning Guide

For any questions, feel free to email me at jason_ho@utexas.edu to debug.

Table of Contents

- [CilialO Fine Tuning Guide](#)
 - [Setting Up Your Environment](#)
 - [Installing Label Studio](#)
 - [Converting Raw Videos to Individual Frames for Label Studio](#)
 - [preprocess_nd2.py](#)
 - [preprocess_tif.py](#)
 - [Example Usage](#)
 - [Launching Label Studio and Using Label Studio](#)
 - [Step 1: Launching label-studio and creating an account](#)
 - [Step 2: Create a New Project](#)
 - [Step 3: Import Your Data](#)
 - [Step 4: Set Up Your Labeling Interface](#)
 - [Step 5: Start Labeling](#)
 - [Step 6: Review and Adjust](#)
 - [Step 7: Export Your Labeled Data](#)
 - [Stopping a Label Studio Session](#)
- [Fine Tuning Ultralytics YOLO](#)
 - [Setup Fine-Tuning Dataset from Label Studio Outputs](#)
 - [Understanding Each Exported .ZIP from Label Studio](#)
 - [Unzipping and Preparing a Dataset of Videos](#)
 - [Running the YOLO Dataset Generation Script](#)
 - [Running YOLO Fine Tuning!](#)
 - [Running Training](#)
 - [Outputs of Ultralytics YOLO Fine-Tuning](#)
 - [Important Files](#)
- [Using the New Fine-Tuned Models in CilialO](#)

Setting Up Your Environment

Make sure to have followed the [cilia_io_user_guide.pdf step 2](#) and [step 3](#) which creates a virtual environment for CilialO in Python and installs all the required libraries necessary for CilialO to function.

To get CilialO ready for fine-tuning, we need to add one more package, [label-studio](#), that will do a lot of the heavy lifting for us for YOLO fine-tuning. The following section details how to install this package.

Installing Label Studio

Note: This only needs to be completed once.

1. Activate your virtual environment. See the [cilia_io_user_guide.pdf](#) for more guidance on loading into your virtual environment.
2. Run the following command:

```
pip install label-studio==1.13.0
```

After it runs, there should be no errors in the terminal.

Note: We ran this with Python 3.10 with label-studio 1.13.0. There have been updates since then, but for exact replication of our flow, please use 1.13.0

Converting Raw Videos to Individual Frames for Label Studio

CiliaO contains two helpful scripts for converting raw .ND2 files and .TIF files into individual .PNG frames for individual annotation of each of the separate frames necessary for Label Studio.

Such files are located at [/src/dataset_creation/preprocess_nd2.py](#) and [/src/dataset_creation/preprocess_tif.py](#), respectively.

The usage of these files is detailed below. We recommend creating a separate output directory for each video file such that the video frames are contained together and not mixed with other frames, which can be confusing in downstream analysis or future analysis.

preprocess_nd2.py

This file is the preprocess script used to preprocess .nd2 zstack files and produce black and white pictures of each frame of the video and save them into a directory. The usage is as follows:

```
usage: preprocess_nd2.py [-h] data_file output_directory

Process ND2 file and save images.

positional arguments:
  data_file             Path to the input ND2 file
  output_directory      Path to the output directory for saving images

options:
  -h, --help            show this help message and exit
```

This is a command line python file that takes in two things:

1. **First argument:** The relative location of the .nd2 file
2. **Second argument:** The folder to save the frames of the specific folder to

preprocess_tif.py

```
usage: preprocess_tif.py [-h] data_file output_directory

Process TIFF file and save images.

positional arguments:
  data_file             Path to the input TIFF file
  output_directory      Path to the output directory for saving images

options:
  -h, --help            show this help message and exit
```

This is a command line python file that takes in two things:

1. **First argument:** The relative location of the .tif file
2. **Second argument:** The folder to save the frames of the specific folder to

Example Usage

I put a file named: *example_cilia_video.nd2* into my cilia project directory. This file, *preprocess_nd2.py* is in the same folder. I want to create a new folder labeled to house all of this information called:

example_cilia_frames with the preprocessing done. I would run the following command:

```
python preprocess_nd2.py example_cilia_video.nd2 example_cilia_frames
```

PLEASE DO NOT MIX DIFFERENT VIDEOS OR RENAME THE FRAMES AS THIS METADATA IS USEFUL FOR FUTURE TRAINING

Launching Label Studio and Using Label Studio

Label-studio is the primary tool that used for the labeling of cilia. Here, there is a nice interface which should be friendly to non-code users that effectively acts as a website that is hosted locally. Here, you will create a project for each video, create the labels, and then label each of the frames in the video by hand. Additionally, you will export all of the data from each labeled video out of this program.

Step 1: Launching label-studio and creating an account

1. After generating a directory from the video file, launch label-studio using the following command in the terminal **while you are still in your virtual environment**:

```
label-studio
```

Make sure not to close the terminal window!

The terminal should hang and state that it is starting a development local server for label studio hosted at 0.0.0.0:8080 as shown below: **Note: If this does not work, MAKE SURE YOU ARE IN YOUR**

VIRTUAL ENVIRONMENT. You will see (virtual_environment_name) in the front of your terminal line. In our case, (cilia_io_venv)

```
May 12, 2026 - 20:27:31
Django version 3.2.25, using settings 'label_studio.core.settings.label_studio'
Starting development server at http://0.0.0.0:8080/
Quit the server with CONTROL-C.
```

2. This call to label studio will create a locally hosted website on your computer. Navigate to your favorite web browser and in the search bar the following hyperlink to navigate to the open label-studio.

<http://localhost:8080/user/login/>

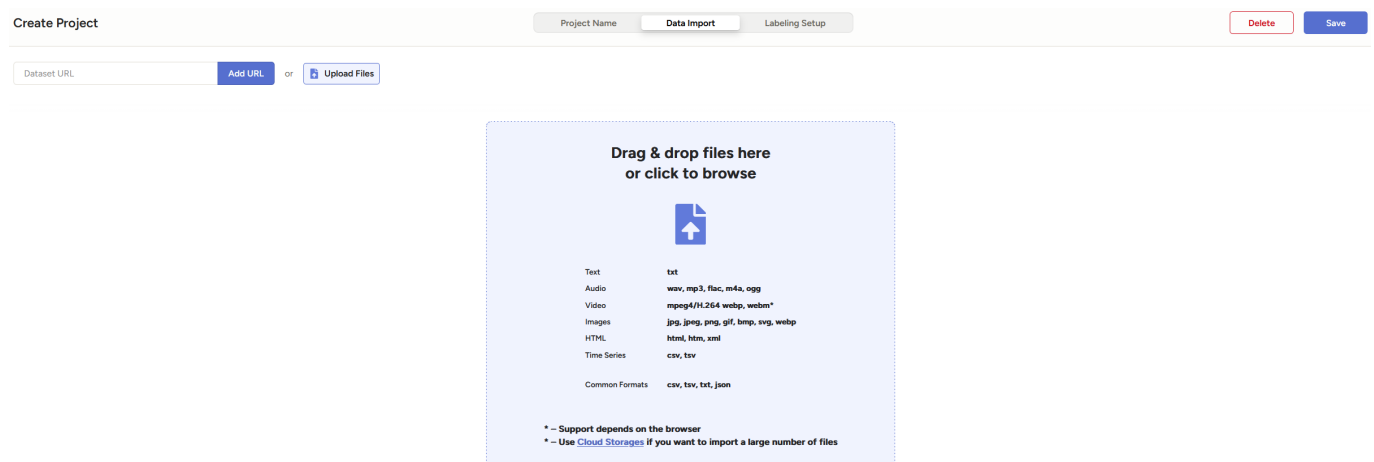
3. If you are a new user, create an account through label studio. Note that this is a local account. The application is hosted entirely on your machine. This means that no confidential data is put on the internet that should not be available.

Step 2: Create a New Project

1. Click on the "Create" button (usually found on the top right hand corner of the dashboard).
2. Give your project a name (e.g., "cilia_io_test_video").

Step 3: Import Your Data

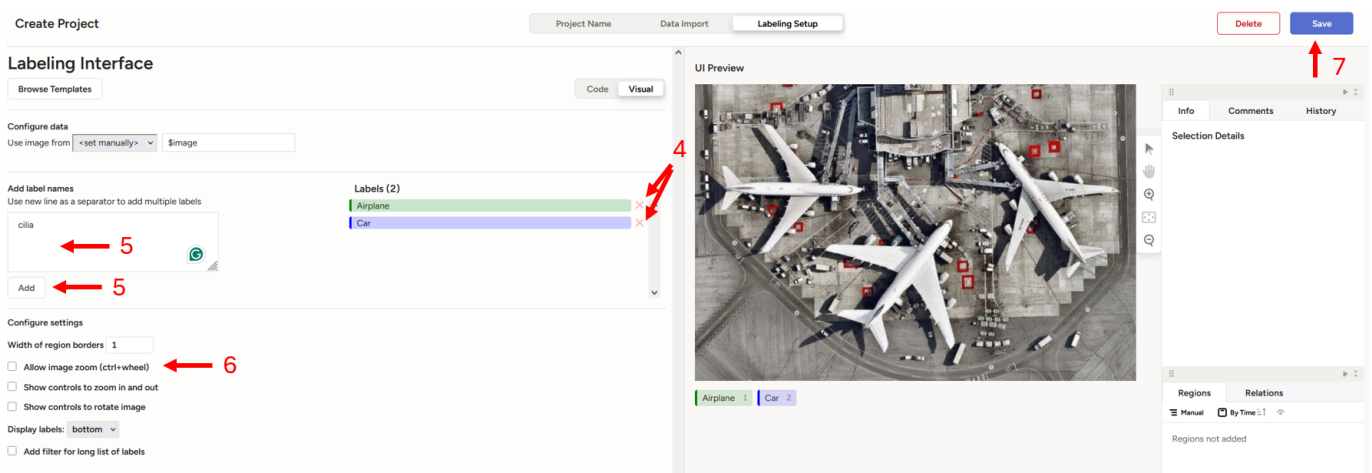
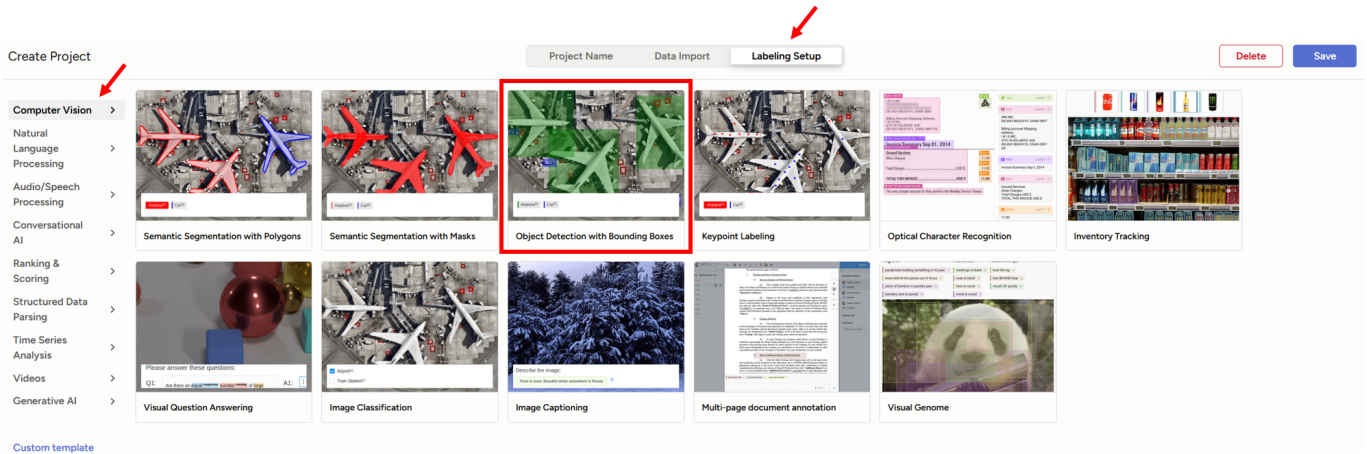
1. In your new project, navigate to the **Data Import** Tab.
2. Click **Upload Files**
3. Select the frames that you want to upload. Note that the project only really supports loading 80 images at a time, so if there are more than 80 frames, you will have to do multiple uploads.
 - For multiple uploads, just do 80 frames at a time (0-79, 80-159, etc.) where after you upload the first 80, just click **Upload More Files** where "Upload Files" button used to be
4. Wait for the upload to complete.



Step 4: Set Up Your Labeling Interface

1. Go to the **Labeling Setup** tab in your new project.
2. Navigate to the **Computer Vision** tab and click **Object Detection with Bounding Boxes**

3. This will bring up a picture of airlines with two labels (airplane and car).
4. Delete the two labels by clicking on the red x next to their names.
5. Under **Add label names** add a label named **cilia** and click add.
6. Check the box that says: **Allow image zoom (ctrl+wheel)** (not necessary, but useful to zoom in)
7. Save your changes.

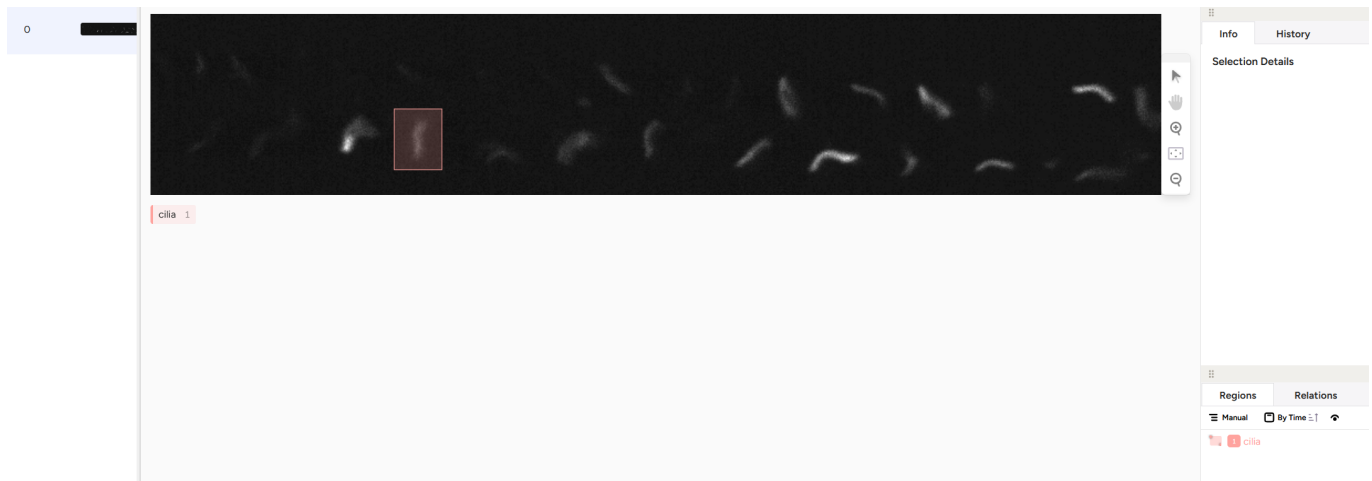


Step 5: Start Labeling

1. Click on **Label All Tasks** in blue
2. You'll see your data items one by one.
3. For each item:
 - Look at the data carefully.
 - On the bottom, click the **cilia** button label which will highlight the cilia button.
 - Left click and drag to make a bounding box for a single cilia
4. Click "Submit" or "Next" to move to the next item.

Note: The data is saved internally after submitting a labeled image. There is no manual saving needed.

Below, we show what one cilia bounding box label looks like in a frame in Label Studio.



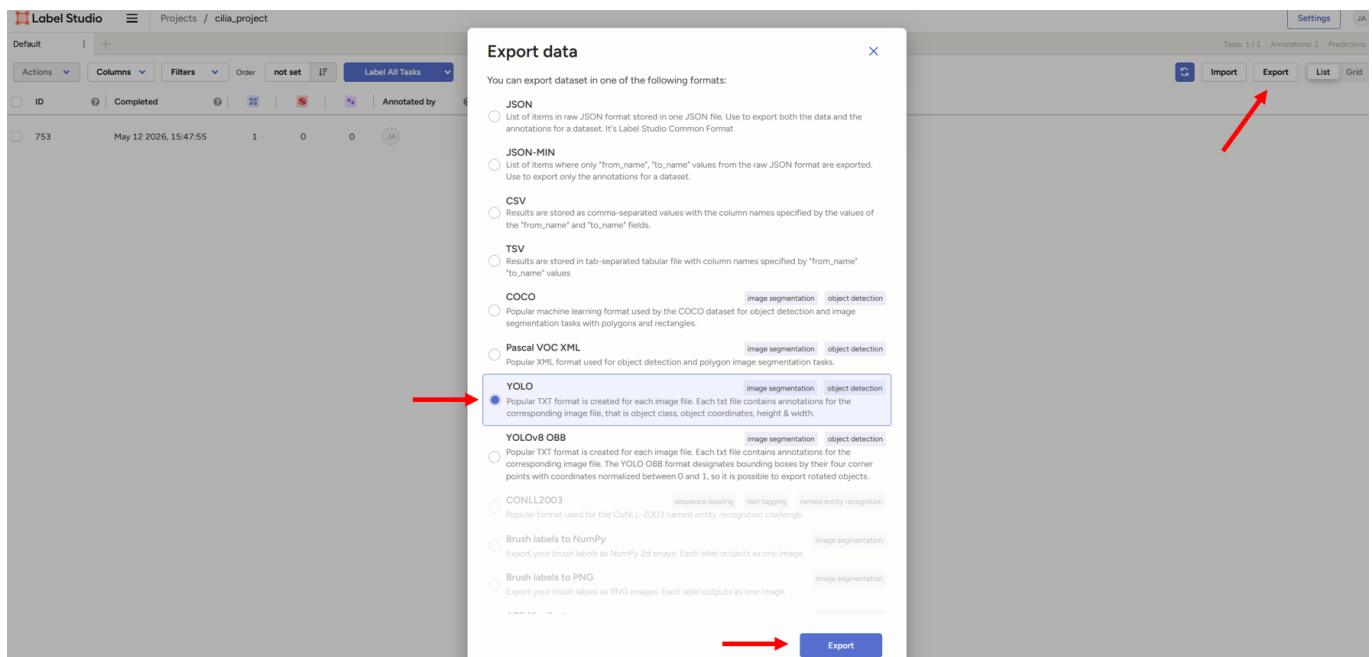
Step 6: Review and Adjust

1. Periodically check your work in the "Results" or "Review" section.
2. Look for any mistakes or inconsistencies in your labeling.
3. You should edit or delete incorrect labels here.

Step 7: Export Your Labeled Data

1. Once you've labeled all the data in one videos, go to the "Export" section.
2. Select **YOLO** as the export configuration
3. Click "Export" and save the file to your computer.
4. This will produce a zip file containing the images, the bounding box labels, and the number of labels in this dataset.

Here is what that screen looks like when exporting. The red arrows indicate the initial export button in the top right-hand corner, the YOLO export configuration, and the Export button.



Stopping a Label Studio Session

- After finishing labeling, you can close out the tab as everything is automatically saved.

- Go to your terminal and close the locally hosted label-studio server with **CTRL + C** on Windows or Linux, or **CMD + C** on Mac
- To open label-studio again, follow the previous instructions to open the virtual environment in the directory where you first installed your virtual environment, and then use the **label-studio** command to startup the server and login with the credentials that you first signed up with!

Fine Tuning Ultralytics YOLO

Setup Fine-Tuning Dataset from Label Studio Outputs

The following section will explain the format of each of the .zip files and how to incorporate these to create a dataset that YOLO can train, test and validate on.

Understanding Each Exported .ZIP from Label Studio

```
dataset.zip
├── images/
│   ├── frame0.png
│   ├── frame1.png
│   └── etc.
├── labels/
│   ├── frame0.txt
│   ├── frame1.txt
│   └── etc.
├── classes.txt
└── notes.json
```

- **images/** — image files
- **labels/** — YOLO annotation .txt files
- **classes.txt** — class names
- **notes.json** — dataset metadata

Each label file corresponds to an image with the same filename.

YOLO annotation format:

<class_id> <x_center> <y_center> <width> <height>

Unzipping and Preparing a Dataset of Videos

If you have multiple labeled videos, unzip all exported folders and place them inside the same parent directory.

Example:

```
data/
├── labeled_data/
```

```
|   ├── video_1/  
|   ├── video_2/  
|   ├── video_3/  
|   └── ...
```

The script, `src/create_yolo_training_dataset.py` will automatically process each video folder and create the YOLO dataset.

In `src/create_yolo_training_dataset.py`, update lines 51–54:

```
TRAIN_RATIO = 0.80  
RAW_DATA_DIR = "../data/labeled_data"  
RAW_TEST_SET_DIR = "../data/cilia_test"  
DATASET_DIR = "../data/cilia_dataset"
```

Variable Descriptions

- **TRAIN_RATIO**: Percentage of videos used for training. Here, 0.80 means 80% training and 20% validation.
- **RAW_DATA_DIR**: Folder containing all labeled video folders for training and validation.
- **RAW_TEST_SET_DIR**: Folder containing the separate test videos. These videos are not used during training.
- **DATASET_DIR**: Output folder where the final YOLO dataset will be created.

Running the YOLO Dataset Generation Script

```
python3 src/create_yolo_training_dataset.py
```

Create a YOLO-formatted dataset inside `DATASET_DIR`

The final dataset structure will look like:

```
DATASET_DIR/  
├── train/  
│   ├── images/  
│   └── labels/  
├── val/  
│   ├── images/  
│   └── labels/  
├── test/  
│   ├── images/  
│   └── labels/  
└── train_val_split.json
```

The script combines all of the training set videos together, all the validation set videos together, and test set videos together and provides a .json file explaining the exact split of everything.

Running YOLO Fine Tuning!

The training script is located at `/yolo/yolo_train.py`

Before running training, update the dataset path inside `/yolo/configs.yaml`. This file looks like this.

Example:

```
path: UPDATE/full/path/to/your/dataset
train: train/images
val: val/images
test: test/images

# Classes
names:
  0: cilia
```

Only update the path field so that it points to your YOLO dataset directory! This is also where we can add updated parameters to the YOLO Ultralytics model, but in order to get the most up to date documentation, check out the YOLO documentation on the Ultralytics website.

Running Training

From the main project directory, navigate to the YOLO directory and run the `yolo_train.py`.

```
cd yolo
python yolo_train.py
```

Outputs of Ultralytics YOLO Fine-Tuning

After training finishes, Ultralytics will create an output folder containing the trained model, plots, and evaluation metrics.

The outputs are usually stored in:

```
runs/detect/train/
```

Example structure:

```
runs/detect/train/
├─ weights/
```

```
| | best.pt  
| | last.pt  
| results.png  
| confusion_matrix.png  
| PR_curve.png  
| F1_curve.png  
| results.csv  
| args.yaml
```

Important Files

- **weights/best.pt**: Best-performing trained model based on validation performance. This is usually the model used for inference/testing.
- **weights/last.pt**: Model weights from the final training epoch.
- **results.png**: Summary plot showing training and validation metrics over time.
- **confusion_matrix.png**: Shows prediction accuracy and common classification errors.
- **PR_curve.png**: Precision–Recall curve for model performance evaluation.
- **F1_curve.png**: F1 score across confidence thresholds.
- **results.csv**: Numerical training and validation metrics for each epoch.
- **args.yaml**: Copy of the training configuration and parameters used for the run.

Using the New Fine-Tuned Models in CiliaIO

1. To incorporate the newly fine-tuned YOLO models, navigate to line 202 in `cilia_io.py` and update the filepath to point to the new checkpointed fine-tuned YOLO model.
2. Run CiliaIO with no other changes!

```
193 # -----  
194  
195 # ML Parameters  
196 if torch.cuda.is_available():  
197     device = "cuda"  
198 elif torch.backends.mps.is_available():  
199     device = "mps"  
200 else:  
201     device = "cpu"  
→ 202 yolo_model_filepath = "../yolo/runs/detect/train/weights/best.pt"  
203  
204 sam_checkpoint = "../sam/sam_vit_h_4b8939.pth"  
205 model_type = "vit_h"  
206 # ---- END HYPERPARAMETERS -----]  
207
```